

Python

Summary Session 11



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Для чего нужен модуль time?



Модуль time

Этот модуль предоставляет функции для работы с временем, включая получение текущего времени, измерение интервалов и управление задержками выполнения кода

Основные функции модуля time: time()



Определение

Возвращает текущее время в секундах, прошедшее с 1 января 1970 года.

Код

```
import time

current_time = time.time()

print(current_time)
```

Основные функции модуля time: sleep(seconds)



Определение

Приостанавливает выполнение программы на указанное количество секунд.

Код

```
import time

print("Программа запущена")

time.sleep(2) # Задержка на 2 секунды

print("2 секунды спустя...")
```


Пример: Измерение времени выполнения программы



```
import time

start_time = time.time()

# Создание объекта range
range_million = range(1000000)
end_time = time.time()
print(f"Время создания range: {end_time - start_time:.10f} секунд")

# Создание объекта списка
lst = [x for x in range(1000000)]
end_time = time.time()
print(f"Время создания list: {end_time - start_time:.10f} секунд")
```

Модуль collections



Модуль `collections` предоставляет дополнительные структуры данных, которые дополняют стандартные типы Python. Эти структуры данных оптимизированы для выполнения различных задач и могут быть более эффективными по сравнению с обычными списками и словарями.

Экспресс-опрос

?

Для чего нужен класс OrderedDict?



Класс OrderedDict

Это класс из модуля `collections`, который представляет собой словарь, сохраняющий порядок добавления элементов. В отличие от стандартных словарей в старых версиях Python (до 3.7), где порядок ключей не гарантировался, `OrderedDict` всегда сохраняет порядок добавления.

Синтаксис



```
from collections import OrderedDict
```

```
ordered_dict = OrderedDict(iterable)
```

Экспресс-опрос

?

Как называется любой итерируемый объект, который предоставляет пары ключ: значение?



iterable

Это любой итерируемый объект, который предоставляет пары ключ: значение (например, список кортежей, словарь, генераторы и т.д.). Если объект не указан, создаётся пустой `OrderedDict`.

Пример создания OrderedDict



```
# Импорт класса из модуля collections
from collections import OrderedDict

# Создание пустого OrderedDict
od = OrderedDict()

# Добавление элементов аналогично работе со словарем
od["a"] = 1
od["b"] = 2
od["c"] = 3

print(od)
```


Реализация OrderedDict на основе словаря



Класс `OrderedDict` реализован на основе встроенного словаря Python. Это означает, что он использует все преимущества стандартного словаря, такие как быстрое хешированное обращение к элементам, но добавляет дополнительную функциональность.

Экспресс-опрос

?

Что делает метод `popitem()` в `OrderedDict`?

Метод popitem()



Метод `popitem()` в `OrderedDict`, как и в `dict`, позволяет удалять и возвращать пары "ключ-значение" из словаря. В отличие от стандартного словаря, `OrderedDict` позволяет указать, с какой стороны забрать значения.

Метод popitem()



Синтаксис

```
key, value =  
ordered_dict.popitem(last=True)
```

Пояснения

last=True – удаляет последний добавленный элемент

last=False – удаляет первый добавленный элемент.

Пример 1. Удаление элементов



```
from collections import OrderedDict
```

od	=	OrderedDict()
od["a"]	=	1
od["b"]	=	2
od["c"]	=	3
od["d"] = 4		

```
# Удаление последнего элемента
print(od.popitem())
print(od)
```

```
# Удаление первого элемента
print(od.popitem(last=False))
print(od)
```

Пример 2. Реализация очереди



```
from collections import OrderedDict
```

queue	=	OrderedDict()
queue["first"]	=	1
queue["second"]	=	2
queue["third"] = 3		

```
# Удаляем элементы с начала очереди
```

```
while queue:
```

```
    print(queue.popitem(last=False))
```

Экспресс-опрос

?

Какой метод позволяет легко перемещать элементы в начало или конец словаря, сохраняя при этом порядок остальных элементов?

Метод `move_to_end`



Метод `move_to_end()` уникален для `OrderedDict` и не доступен в стандартных словарях Python. Его функциональность позволяет легко перемещать элементы в начало или конец словаря, сохраняя при этом порядок остальных элементов.

Метод move_to_end



Синтаксис

```
ordered_dict.move_to_end(key,  
last=True)
```

Пояснение

- key — ключ элемента, который нужно переместить.
- last=True (по умолчанию) указывает, что элемент будет перемещён в конец.
- last=False указывает, что элемент будет перемещён в начало.

Пример применения



```
from collections import deque
queue = deque()
queue["task1"] = "low priority"
queue["task2"] = "medium priority"
queue["task3"] = "low priority"
queue["task4"] = "high priority"
queue["task5"] = "medium priority"

# Собираем ключи для перемещения
keys_to_end = [key for key, value in queue.items() if "low" in value]
keys_to_start = [key for key, value in queue.items() if "high" in value]

# Перемещаем задачи с низким приоритетом в конец
for key in keys_to_end:
    queue.move_to_end(key)

# Перемещаем задачи с высоким приоритетом в начало
for key in keys_to_start:
    queue.move_to_end(key, last=False)

print(queue)
```

Экспресс-опрос

?

Назовите ключевую особенность класса `defaultdict`



defaultdict

Это класс из модуля `collections`, который расширяет возможности стандартного словаря Python. Его ключевая особенность — возможность задавать значение по умолчанию для отсутствующих ключей, что упрощает работу с данными и уменьшает вероятность возникновения ошибок `KeyError`

Класс defaultdict



Синтаксис

```
from collections import defaultdict
```

```
defaultdict(default_type)
```

Пояснение

- default_type — это функция или класс, который автоматически создаёт значение **по умолчанию** для отсутствующего ключа

Особенности и ограничения defaultdict

1

Сохранение значения по умолчанию

После обращения к отсутствующему ключу он автоматически добавляется в словарь со значением по умолчанию

2

Совместимость

defaultdict полностью совместим с методами стандартного словаря Python

3

Отсутствие default_type

Если при создании не указан default_type, defaultdict ведёт себя как обычный словарь и вызывает KeyError при доступе к отсутствующему ключу

Пример для сохранения значения по умолчанию



```
from collections import defaultdict

dd = defaultdict(int)

print(dd['missing']) # Добавлено значение 0

print(dd)
```

Пример 1. Создание словаря со значением типа `int` по умолчанию

```
from collections import defaultdict

# Словарь с числовым значением по умолчанию

dd = defaultdict(int)

# Присваивает ключу базовое значение int

print(dd['a'])
```

1

```
# Обновляет имеющийся ключ

dd['a'] += 1

print(dd['a'])

# Присваивает ключу базовое значение int и обновляет его

dd['b'] += 10

print(dd['b'])

print(dd)
```

2

Пример 2. Создание словаря со списком по умолчанию

```
from collections import defaultdict
```

```
# Словарь с пустым списком по умолчанию
```

```
dd = defaultdict(list)
```

```
# Присваивает ключу базовое значение list  
и обновляет его
```

```
dd['a'].append(1)
```

1

```
# Обновляет список по имеющемуся ключу
```

```
dd['a'].append(2)
```

```
# Присваивает ключу базовое значение list  
и обновляет его
```

```
dd['b'].append(10)
```

```
print(dd)
```

2

Пример 2. Использование кастомной функции

```
from collections import defaultdict

# Пользовательская функция для значения по умолчанию
def default_value():
    return "default"

dd = defaultdict(default_value)

print(dd['missing_key'])

print(dd)
```

Экспресс-опрос

?

Назовите три преимущества `defaultdict`

Преимущества defaultdict

1

Упрощение кода

Нет необходимости
проверять
существование ключа
перед изменением
значения

2

Уменьшение количества
ошибок

Исключает вероятность
возникновения `KeyError`
при доступе к
отсутствующему ключу

3

Гибкость

Можно задать любое
значение по умолчанию,
используя функцию или
класс

Пример упрощения кода



Обычный словарь

```
my_dict = {}

if 'a' not in my_dict:
    my_dict['a'] = []

my_dict['a'].append(1)
```

defaultdict

```
from collections import defaultdict

dd = defaultdict(list)

dd['a'].append(1)
```

Экспресс-опрос

?

Назовите класс из модуля `collections`, предназначенный для подсчёта количества элементов в итерируемом объекте



Counter

Это класс из модуля `collections`, предназначенный для подсчёта количества элементов в итерируемом объекте. Он автоматически создаёт словарь, где элементы — это ключи, а их количество — значения.

Класс Counter



Синтаксис

```
from collections import Counter
```

```
Counter(iterable)
```

```
Counter(mapping)
```

```
Counter(**kwargs)
```

Пояснение

- `iterable` — итерируемый объект (строка, список и т.д.), элементы которого нужно подсчитать
- `mapping` — словарь, где ключи — элементы, а значения — их количество
- `kwargs` — произвольные пары ключ-значение

Основные особенности класса Counter



Автоматический подсчёт элементов



Поддержка стандартных операций со словарями



Поддержка арифметических операций

Пример 1. Подсчёт символов в строке

```
from collections import Counter
```

```
text = "hello world"
```

```
counter = Counter(text)
```

```
print(counter)
```

Пример 2. Подсчёт слов в списке

```
from collections import Counter
```

```
words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
```

```
counter = Counter(words)
```

```
print(counter)
```

Пример 3. Подсчёт слов в списке

```
from collections import Counter

data = {"apple": 3, "banana": 2, "cherry": 1}
counter = Counter(data)
print(counter)
```

Экспресс-опрос

?

Какие методы класса counter вы помните?

Метод `most_common([n])`



Описание

- Возвращает список из `n` наиболее часто встречающихся элементов, отсортированных по убыванию.
- Если `n` не указано, возвращаются все элементы.

Пример

```
counter = Counter("banana")
print(counter.most_common(2))
```

Метод elements()



Описание

- Возвращает итератор, который повторяет элементы столько раз, сколько они встречаются
- Если элемент имеет отрицательное или нулевое количество, он игнорируется

Пример

```
counter = Counter({"a": 3, "b": 1, "c": 0})
iter_count = counter.elements()

print(list(iter_count))
```

Метод `subtract([iterable-or-mapping])`



Описание

- Вычитает элементы, уменьшая их количество. Может принимать как итерируемый объект, так и словарь
- Значения могут стать отрицательными

Пример

```
counter = Counter("banana")
counter.subtract("an")
print(counter)
```


Метод update([iterable-or-mapping])



Описание

- Увеличивает количество элементов из переданного объекта

Пример

```
counter = Counter("banana")
counter.update("nan")
print(counter)
```

Экспресс-опрос

?

Перечислите возможные операции между объектами counter

Сложение



Описание

Объединяет два Counter, складывая количества одинаковых элементов

Пример

```
c1 = Counter("banana")
c2 = Counter("apple")
print(c1 + c2)
```

Вычитание



Вычитает количества, игнорируя отрицательные результаты

Пример

```
c1 = Counter("banana")
c2 = Counter("an")
print(c1 - c2)
```

Пересечение



Оставляет минимальные количества
одинаковых элементов

Пример

```
c1 = Counter("banana")
c2 = Counter("an")
print(c1 & c2)
```

Объединение



Оставляет максимальные количества
одинаковых элементов

Пример

```
c1 = Counter("banana")
c2 = Counter("an")
print(c1 | c2)
```

Экспресс-опрос

?

Можно ли функции в Python передавать в другие функции?

Передача функций в качестве аргументов



Функции в Python являются объектами, которые можно присваивать переменным, хранить в коллекциях и передавать в другие функции.

Как это работает?



Не нужно указывать () при передаче, иначе передастся результат её выполнения.



Функция может быть вызвана внутри другой функции по переданной ссылке с помощью ().

Передача функции без вызова



```
def square(x):  
    return x * x  
  
def cube(x):  
    return x * x * x  
  
def apply_function(func, value):  
    return func(value) # Вызываем переданную функцию внутри другой функции  
  
result_square = apply_function(square, 5) # Передаём функцию square без вызова (без скобок)  
result_cube = apply_function(cube, 5) # Передаём функцию cube без вызова (без скобок)  
  
print(result_square)  
  
print(result_cube)
```

Передача встроенной функции



```
# Можно передавать не только пользовательские функции, но и встроенные
result_abs = apply_function(abs, -10)
print(result_abs)
result_len = apply_function(len, "-10")
print(result_len)
```

Ошибка при передаче вызванной функции



```
# Передается результат вызова функции, а не ссылка на функцию  
result = apply_function(square(5), 5) # Ошибка!
```

Хранение функций в коллекциях



```
def add(x, y):
    return x + y

def multiply(x, y):
    return x * y

# Функции можно хранить в списках, словарях и передавать их динамически
operations = {
    "+": add,
    "*": multiply
}

choice = input("Выберите операцию (+, *): ")

# Из словаря получена функция и скобки с аргументами запускают её
print(operations[choice](10, 5))
```

Экспресс-опрос

?

Что такое Lambda-функции?



Lambda-функция

Это небольшая, одноразовая функция, которая не требует явного объявления с `def`. Она используется для краткой записи простых операций и может быть передана как аргумент в другие функции.

Lambda-функции



Пояснение

- `lambda` — ключевое слово для создания анонимной функции.
- `arguments` — параметры, которые принимает функция.
- `expression` — выражение, результат которого возвращается (без `return`).

Синтаксис

`lambda arguments: expression`

Lambda с одним аргументом



Функция принимает число x и возвращает его квадрат

```
square = lambda x: x ** 2
```

```
print(square(4))
```

```
print(square(5))
```

Аналог с def

```
def square(x):
```

```
    return x ** 2
```

```
print(square(4))
```

```
print(square(5))
```

Lambda с несколькими аргументами



Функция принимает два аргумента и возвращает их сумму

```
add = lambda x, y: x + y
```

```
print(add(3, 5))
```

```
print(add(8, 9))
```

Аналог с def

```
def add(x, y):
```

```
    return x + y
```

```
print(add(3, 5))
```

```
print(add(8, 9))
```

Lambda как аргументы других функций



Lambda-функции можно передавать как аргументы в другие функции, не создавая отдельные именованные функции.

Lambda как аргументы других функций



```
def apply_func(func, numbers):  
    return [func(num) for num in numbers]  
  
result = apply_func(lambda x: x + 10, [5, 8, 3])  
print(result)
```

Экспресс-опрос

?

Перечислите особенности Lambda-функций

Особенности lambda-функций



- Lambda-функция всегда возвращает результат выражения
- Lambda может содержать только одно выражение
- Нет многострочных блоков кода (if, for и т. д.)

Экспресс-опрос

?

Что такое парадигма программирования?



Парадигма программирования

Это стиль написания и организации кода, который определяет способы решения задач в программировании.

Экспресс-опрос

?

Какие парадигмы программирования вы помните?

Основные парадигмы программирования

Императивное

Процедурное

Объектно-
ориентированное

Функциональное

Декларативное

Императивное программирование



Описывает последовательность команд, которые изменяют состояние программы.



Код состоит из инструкций, выполняемых шаг за шагом.



Пример языков: Python, C, Java.

Процедурное программирование



Код организован в функции (процедуры), каждая из которых выполняет определённую задачу.



Используется разделение программы на логические блоки для повторного использования.



Пример: Python, Pascal, C.

Объектно-ориентированное программирование



Основной концепцией является объект, который объединяет данные и методы для их обработки.



Код организуется в классы и объекты, позволяя моделировать реальные сущности.



Пример: Python, Java, C++.

Функциональное программирование



Основано на функциях высшего порядка, чистых функциях и отсутствии изменения состояния.



Предпочитает использование неизменяемых данных и рекурсии вместо циклов.



Пример: Haskell, Lisp, Python.

Декларативное программирование



Описывает **что** должно быть сделано, а не **как**.



Включает функциональное программирование, а также языки разметки (SQL, HTML).



Пример: SQL, Prolog, Haskell.

Современные языки программирования



Такие как Python, поддерживают несколько парадигм, позволяя использовать гибридные подходы. Например, в Python можно писать как в процедурном стиле, так и использовать ООП и функциональные концепции. Выбор парадигмы зависит от задачи, удобства и требований проекта.



Функциональное программирование

Это парадигма программирования, в которой основной единицей организации кода являются функции. Программы строятся из чистых функций, которые принимают аргументы и возвращают результат, не изменяя состояние программы.

Экспресс-опрос

?

Перечислите основные принципы функционального программирования

Основные принципы функционального программирования



Чистые функции



Неизменяемость данных



Функции как объекты



Функции высшего порядка



Рекурсия вместо циклов

Экспресс-опрос

?

Что такое функции высшего порядка?



Функции высшего порядка

Это функции, которые могут принимать другие функции в качестве аргументов и/или возвращать функции в качестве результата

Признаки функций высшего порядка



Принимают функции в качестве аргументов



Возвращают функции как результа

Часто используемые функции высшего порядка



Встроенные функции высшего порядка: `map()`, `filter()`, `reduce()`



Использование `sorted()` с ключом сортировки

Функции map, filter и reduce



Это функции высшего порядка, которые принимают другую функцию в качестве аргумента и применяют её к элементам переданного итерируемого объекта

Экспресс-опрос

?

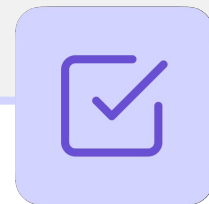
Что делает функция `map()`?



Функция `map()`

Эта функция применяет переданную функцию к каждому элементу одного или нескольких итерируемых объектов и возвращает итератор с результатами

Особенности функции map



- Используется для преобразования данных
- Результат нужно преобразовать в список или другой итерируемый объект, чтобы увидеть значения

Функция map()



Синтаксис

```
map(function, iterable)
```

Пояснения

- `function` — функция, применяемая к каждому элементу
- `iterable` — итерируемый объект (список, кортеж, строка и т.д.)

Пример с одним объектом



```
numbers = [1, 2, 3, 4]

# Каждый элемент списка возводится в квадрат

squared = map(lambda x: x ** 2, numbers)

print(list(squared)) # [1, 4, 9, 16]
```

Пример с несколькими объектами



```
a = [1, 2, 3]
```

```
b = [4, 5, 6]
```

```
# Каждая пара элементов списков суммируется
```

```
result = map(lambda x, y: x + y, a, b)
```

```
print(list(result)) # [5, 7, 9]
```

Пример со встроенными функциями



```
group_numbers = [(1, 2, 3), (4, 5, 6), (7, 8, 9)]
```

```
# К каждому кортежу применяется функция sum
```

```
result = map(sum, group_numbers)
```

```
print(list(result)) # [6, 15, 24]
```

Экспресс-опрос

?

Что делает функция `filter()`?



Функция filter()

Эта функция используется для фильтрации элементов итерируемого объекта, возвращая только те, для которых переданная функция-предикат возвращает True

Особенности функции filter



- Используется для отбора нужных элементов, соответствующих условию
- Если вместо функции передать None, будут выбраны только элементы, которые оцениваются как True
- Результат нужно преобразовать в список или другой итерируемый объект, чтобы увидеть значения

Функция filter()



Синтаксис

```
filter(function, iterable)
```

Пояснения

- `function` — функция-предикат, определяющая условие для фильтрации
- `iterable` — итерируемый объект

Пример с функцией-предикатом



```
numbers = [1, 2, 4, 5, 7, 9, 10, 11]

# Из списка выбираются только чётные числа

even_numbers = filter(lambda x: x % 2 == 0, numbers)

print(list(even_numbers)) # [2, 4, 10]
```

Пример с None



```
data = [0, 1, False, True, '', 'Python', [], [1, 2, 3]]
# Из списка выбираются только те элементы, которые оцениваются как True
true_list = filter(None, data)
print(true_list)
print(list(true_list))
```

Экспресс-опрос

?

Что делает функция `reduce()`?



Функция `reduce()`

Эта функция последовательно применяет переданную функцию к элементам итерируемого объекта с накоплением результата, сводя его к одному значению. Она находится в модуле `functools`

Особенности функции reduce



- Используется для агрегации данных (например, суммы или произведения).
- Работает с парой элементов, начиная с первых двух, и продолжает с результатом и каждым следующим значением

Функция reduce()



Синтаксис

```
from functools import reduce
reduce(function, iterable, initializer)
```

Пояснения

- `function` — функция, принимающая два аргумента и возвращающая один результат
- `iterable` — итерируемый объект
- `initializer` (необязательно) — начальное значение для накопления результата

Пример



```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
# Умножение всех элементов списка последовательно
```

```
result = reduce(lambda x, y: x * y, numbers)
```

```
print(result) # 24
```

Пример с initializer



```
from functools import reduce

numbers = [1, 2, 3, 4]

# Умножение всех элементов списка, начиная с 10
result = reduce(lambda x, y: x * y, numbers, 10)

print(result) # 240
```

Сравнение функций map, filter, reduce

Функция	Описание	Возвращает
map	Применяет функцию ко всем элементам итерируемого объекта	Итератор преобразованных элементов
filter	Фильтрует элементы на основе условия	Итератор отфильтрованных элементов
reduce	Последовательно применяет функцию, накапливая результат	Итоговое значение

Экспресс-опрос

?

Что считается ложным значением?

Функции any и all



Ложные значения

0, None, False, пустые коллекции ([], {}, ()), set())

Истинные значения

Всё, что не считается ложным

Экспресс-опрос

?

Что делает функция `any()`?



Функция `any()`

Эта функция проверяет, содержит ли итерируемый объект хотя бы один истинный элемент

Функция any()



Синтаксис

`any(iterable)`

Пояснения

- `iterable` — итерируемый объект (список, кортеж, строка, множество, словарь и т. д.), элементы которого нужно проверить

Особенности функции any



- Возвращает True, если хотя бы один элемент в итерируемом объекте является истинным.
- Если все элементы ложные, возвращает False.
- Если объект пустой, возвращает False.
- Выполняет проверку поэлементно и останавливается, как только находит истинное значение (**ленивое вычисление**)

Пример использования



```
data = [0, None, False, 1]
```

```
print(any(data)) # True, так как есть хотя бы одно истинное значение (1)
```

```
data = [0, None, False]
```

```
print(any(data)) # False, так как все элементы ложные
```

```
data = []
```

```
print(any(data)) # False, так как объект пустой
```

Экспресс-опрос

?

Что делает функция `all()`?



Функция all()

Эта функция проверяет, являются ли все элементы итерируемого объекта истинными

Функция all()



Синтаксис

```
all(iterable)
```

Пояснения

- `iterable` — итерируемый объект, элементы которого нужно проверить

Особенности функции all



- Возвращает True, если **все** элементы в итерируемом объекте истинные
- Если хотя бы один элемент ложный, возвращается False
- Возвращает True для **пустого объекта**, так как нет элементов, которые можно было бы считать ложными
- Выполняет проверку поэлементно и останавливается, как только находит ложное значение

Пример использования



```
data = [1, 2, 3]
print(all(data)) # True, так как все элементы истинные
```

```
data = [1, 0, 3]
print(all(data)) # False, так как 0 – ложное значение
```

```
data = []
print(all(data)) # True, так как объект пустой
```


Сравнение функций any и all

Характеристика	any	all
Результат	True, если хотя бы один элемент истинный.	True, если все элементы истинные.
Пустой объект	Возвращает False.	Возвращает True.
Остановка проверки	При нахождении первого истинного значения.	При нахождении первого ложного значения.
Применение	Проверка наличия истинных значений.	Проверка, что все значения истинные.

Пример использования



Проверка, что хотя бы один объект соответствует условию

```
conditions = [x > 10 for x in [5, 20, 8]]
```

```
print(any(conditions)) # True
```

Проверка, что все объекты соответствуют условию

```
conditions = [x > 0 for x in [5, 20, 8]]
```

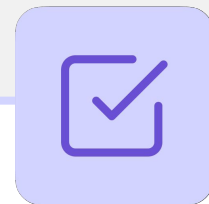
```
print(all(conditions)) # True
```

Экспресс-опрос

?

Для чего нужен параметр `key()`?

Параметр **key**



В Python функции `sorted()`, `min()` и `max()` принимают необязательный параметр **key**.

Он определяет, по какому критерию выполнять сортировку или поиск минимального/максимального значения.

В параметр `key` можно передавать встроенные и пользовательские функции

Экспресс-опрос

?

Что делает функция `sorted()`?

Функция sorted



Пояснения

Эта функция сортирует элементы по значению, которое возвращает переданная функция

Синтаксис

```
sorted(iterable, key=function,  
reverse=False)
```

Пример: использование встроенной функции



Код

```
words = ['mango', 'grape', 'apple',  
'strawberry', 'banana', 'pineapple',  
'kiwi', 'blueberry']
```

Сортировка по длине слов

```
result = sorted(words, key=len)
```

```
print(result)
```

Вывод

```
['kiwi', 'apple', 'mango', 'grape',  
'banana', 'orange', 'pineapple',  
'strawberry']
```

Пример: пользовательская функция



```
def last_char_len(s):  
    return s[-1], len(s)  
  
words = ['mango', 'grape', 'apple', 'strawberry', 'banana', 'pineapple', 'kiwi',  
         'blueberry']  
  
# Сортировка по последнему символу и длине слова  
result = sorted(words, key=last_char_len)  
  
print(result)
```


Пример: анонимная функция lambda



```
words = ['mango', 'grape', 'apple', 'Strawberry', 'Banana', 'pineapple', 'kiwi', 'blueberry']
```

```
# Сортировка по первому символу (игнорируя регистр) и по последнему символу
```

```
result = sorted(words, key=lambda x: (x[0].lower(), x[-1]))
```

```
print(result)
```

Пример: сортировка списка кортежей



Код

```
students = [("Alice", 25), ("Bob", 20),  
            ("Charlie", 23)]
```

```
# Сортировка списка кортежей по возрасту  
# (второй элемент)
```

```
sorted_students = sorted(students,  
                          key=lambda x: x[1])
```

```
print(sorted_students)
```

Вывод

```
[('Bob', 20), ('Charlie', 23), ('Alice',  
25)]
```

Экспресс-опрос

?

Что делают функции `min()` и `max()`?

Функции min() и max()



Пояснения

Функции `min()` и `max()` с параметром `key` позволяют находить минимальный или максимальный элемент на основе вычисленного значения

Синтаксис

```
min(iterable, key=function)
```

```
max(iterable, key=function)
```

Пример: поиск самого длинного слова



Код

```
words = ["apple", "banana", "kiwi",  
"grapefruit"]  
  
longest_word = max(words, key=len)  
  
print(longest_word)
```

Вывод

```
grapefruit
```

Пример: поиск города с минимальным населением



Код

```
cities = [('New York', 8419600), ('Los
Angeles', 3980400), ('Chicago',
2716000)]

smallest_city = min(cities, key=lambda
x: x[1])

print(smallest_city)
```

Вывод

```
('Chicago', 2716000)
```



ВОПРОСЫ





ЗАДАНИЕ



Задание 1

Какой результат будет выведен при выполнении следующего кода?

```
def process_data(func, data):
    return [func(i) for i in data]

numbers = [1, 2, 3]
result = process_data(lambda x: x * 2, numbers)
print(result)
```

- a. [1, 2, 3]
- b. [2, 4, 6]
- c. Ошибка

Задание 1

Какой результат будет выведен при выполнении следующего кода?

```
def process_data(func, data):
    return [func(i) for i in data]
```

```
numbers = [1, 2, 3]
result = process_data(lambda x: x * 2, numbers)
print(result)
```

- a. [1, 2, 3]
- b. [2, 4, 6]
- c. Ошибка

Задание 2



Можно ли использовать `if/else` в `lambda`-функции?

Задание 2

Можно ли использовать if/else в lambda-функции?

Ответ: Да, но только в виде тернарного оператора

Пример:

```
min_value = lambda a, b: a if a < b else b
print(min_value(3, 7))
```

Задание 3

Какой результат будет выведен при выполнении следующего кода?

```
def apply_function(func, data):
    return [func(x) for x in data]
```

```
result = apply_function(lambda x: x + 5, [5, 8, 3])
print(result)
```

- a. [5, 8, 3]
- b. [10, 15, 20]
- c. [10, 13, 8]
- d. Ошибка

Задание 3

Какой результат будет выведен при выполнении следующего кода?

```
def apply_function(func, data):
    return [func(x) for x in data]
```

```
result = apply_function(lambda x: x + 5, [5, 8, 3])
print(result)
```

- a. [5, 8, 3]
- b. [10, 15, 20]
- c. [10, 13, 8]
- d. Ошибка

Задание 4

Какой результат будет выведен при выполнении следующего кода?

```
from functools import reduce
```

```
numbers = [2, 3, 4]
```

```
result = reduce(lambda x, y: x * y, numbers, 5)
```

```
print(result)
```

- a. 9
- b. 120
- c. 24
- d. 84

Задание 4

Какой результат будет выведен при выполнении следующего кода?

```
from functools import reduce
```

```
numbers = [2, 3, 4]
```

```
result = reduce(lambda x, y: x * y, numbers, 5)
```

```
print(result)
```

a. 9

b. 120

c. 24

d. 84

Задание 5

Какой результат будет выведен при выполнении следующего кода?

```
data = [0, None, False, "", []]  
print(any(data))
```

- a. True
- b. False
- c. None

Задание 5

Какой результат будет выведен при выполнении следующего кода?

```
data = [0, None, False, "", []]
print(any(data))
```

- a. True
- b. False
- c. None

Задание 6

Какой результат будет выведен при выполнении следующего кода?

```
data = [True, False, True]  
print(any(data) and all(data))
```

- a. True
- b. False
- c. None

Задание 6

Какой результат будет выведен при выполнении следующего кода?

```
data = [True, False, True]
print(any(data) and all(data))
```

- a. True
- b. False
- c. None

Задание 7

Какой результат будет выведен при выполнении следующего кода?

```
words = ["apple", "banana", "kiwi", "grapefruit"]
shortest_word = min(words, key=len)
print(shortest_word)
```

- a. apple
- b. banana
- c. kiwi
- d. grapefruit

Задание 7

Какой результат будет выведен при выполнении следующего кода?

```
words = ["apple", "banana", "kiwi", "grapefruit"]
shortest_word = min(words, key=len)
print(shortest_word)
```

- a. apple
- b. banana
- c. kiwi
- d. grapefruit

Задание 8

Какой результат будет выведен при выполнении следующего кода?

```
words = ["apple", "Banana", "kiwi", "grapefruit"]
sorted_words = sorted(words, key=lambda x: x.lower())
print(sorted_words)
```

- a. ['apple', 'Banana', 'grapefruit', 'kiwi']
- b. ['Banana', 'apple', 'grapefruit', 'kiwi']
- c. ['kiwi', 'grapefruit', 'apple', 'Banana']
- d. ['apple', 'Banana', 'kiwi', 'grapefruit']

Задание 8

Какой результат будет выведен при выполнении следующего кода?

```
words = ["apple", "Banana", "kiwi", "grapefruit"]
sorted_words = sorted(words, key=lambda x: x.lower())
print(sorted_words)
```

- a. ['apple', 'Banana', 'grapefruit', 'kiwi']
- b. ['Banana', 'apple', 'grapefruit', 'kiwi']
- c. ['kiwi', 'grapefruit', 'apple', 'Banana']
- d. ['apple', 'Banana', 'kiwi', 'grapefruit']

Задание 9

Какой результат будет выведен при выполнении следующего кода?

```
students = [("Alice", 25), ("Bob", 20), ("Charlie", 23)]  
oldest_student = max(students, key=lambda x: x[1])  
print(oldest_student)
```

- a. ("Alice", 25)
- b. ("Bob", 20)
- c. ("Charlie", 23)
- d. ("Bob", 25)

Задание 9

Какой результат будет выведен при выполнении следующего кода?

```
students = [("Alice", 25), ("Bob", 20), ("Charlie", 23)]  
oldest_student = max(students, key=lambda x: x[1])  
print(oldest_student)
```

- a. ("Alice", 25)
- b. ("Bob", 20)
- c. ("Charlie", 23)
- d. ("Bob", 25)

Задание 10

Какой результат будет выведен при выполнении следующего кода?

```
students = [("Alice", 25), ("Bob", 20), ("Charlie", 23)]
sorted_students = sorted(students, key=lambda x: x[1], reverse=True)
print(sorted_students)
```

- a. [('Charlie', 23), ('Alice', 25), ('Bob', 20)]
- b. [('Alice', 25), ('Charlie', 23), ('Bob', 20)]
- c. [('Bob', 20), ('Charlie', 23), ('Alice', 25)]
- d. [('Charlie', 23), ('Bob', 20), ('Alice', 25)]

Задание 10

Какой результат будет выведен при выполнении следующего кода?

```
students = [("Alice", 25), ("Bob", 20), ("Charlie", 23)]
sorted_students = sorted(students, key=lambda x: x[1], reverse=True)
print(sorted_students)
```

- a. [('Charlie', 23), ('Alice', 25), ('Bob', 20)]
- b. [('Alice', 25), ('Charlie', 23), ('Bob', 20)]
- c. [('Bob', 20), ('Charlie', 23), ('Alice', 25)]
- d. [('Charlie', 23), ('Bob', 20), ('Alice', 25)]

Задание 11

Что делает параметр `key` в функциях `sorted()`, `min()` и `max()`?

Задание 11



Что делает параметр `key` в функциях `sorted()`, `min()` и `max()`?

Ответ: Он определяет **критерий**, по которому выполняется сортировка или поиск минимального/максимального значения.

Задание 12



Можно ли передавать в `sorted()` несколько критериев сортировки?

Задание 12

Можно ли передавать в `sorted()` несколько критериев сортировки?

Ответ: Да, используя кортеж

Пример:

```
words = ["Apple", "banana", "Cherry"]
sorted_words = sorted(words, key=lambda x: (x.lower(), x))
```

Это сортирует по **регистронезависимому** порядку, а затем по алфавиту.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

Повторения букв

Реализуйте функцию, которая принимает текст и возвращает словарь с подсчётом количества каждой буквы, игнорируя регистр.

Данные:

```
text = "Programming is fun!"
```

Пример вывода:

```
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 2, 'n': 2, 's': 1, 'f': 1, 'u': 1}
```

Домашнее задание

Группировка студентов по классам

Создайте структуру для группировки студентов по классам.

Добавьте студентов в соответствующие группы.

Данные:

```
students = [("class1", "Alice"), ("class2", "Bob"), ("class1", "Charlie"), ("class3", "Daisy")]
```

Пример вывода:

```
{'class1': ['Alice', 'Charlie'], 'class2': ['Bob'], 'class3': ['Daisy']}
```

Домашнее задание

Выбор заказов

У вас есть список заказов. Каждый заказ содержит название продукта и его цену. Напишите функцию, которая:

1. Отбирает заказы дороже 500.
2. Создаёт список названий отобранных продуктов в алфавитном порядке.
3. Возвращает итоговый список названий.

Данные:

```
orders = [  
    {"product": "Laptop", "price": 1200},  
    {"product": "Mouse", "price": 50},  
    {"product": "Keyboard", "price": 100},  
    {"product": "Monitor", "price": 300},  
    {"product": "Chair", "price": 800},  
    {"product": "Desk", "price": 400}  
]
```

Пример вывода:

```
['Chair', 'Laptop']
```

Домашнее задание

Статистика продаж

Дан список продаж в виде кортежей (товар, количество, цена).

Напишите программу, которая:

1. Вычисляет общую выручку для каждого товара.
2. Возвращает словарь с товарами {товар: выручка}, отсортированный по убыванию выручки.

Данные:

```
sales = [  
    ("Laptop", 5, 1200),  
    ("Mouse", 50, 20),  
    ("Keyboard", 30, 50),  
    ("Monitor", 10, 300),  
    ("Chair", 20, 800)  
]
```

Пример вывода:

```
{'Chair': 16000, 'Laptop': 6000,  
'Monitor': 3000, 'Keyboard': 1500,  
'Mouse': 1000}
```

Заключение

